TEST CASE DOCUMENT

Pytestify Studio – AI Powered Postman to Pytest Migration Platform

1. Introduction

1.1 Purpose

This document validates the functionality of the Pytestify Studio platform — an AI-powered system that automates conversion of Postman Collections into executable Python pytest test suites, with execution monitoring, AI analysis, audit logging, authentication, and role-based access control.

1.2 Scope

- User Authentication
- Role-Based Access Control (RBAC)
- Project Management
- Collection Import
- Framework Configuration
- Pytest Code Generation
- AI Analysis
- Execution Center
- Audit Logging
- Login History Tracking
- Access Restriction Validation

1.3 Objectives

1. Validate user authentication
2. Verify project creation and management
3. Ensure Postman collections are imported correctly
4. Validate AI-powered pytest generation
5. Verify execution workflow
6. Ensure audit logs and login history are recorded
7. Validate role-based access restrictions

2. Test Environment

Parameter	Value
Operating System	Windows 11
Frontend	React + TypeScript
Backend	Node.js + Express
Database	Supabase PostgreSQL
IDE	Visual Studio Code
Browser	Google Chrome
AI Engine	Gemini AI
Output	Python Pytest Code

3. Test Cases

TC_001	User Login Validation	PASS
Objective	Validate successful user authentication.	
Input	Username: staff | Password: staff	
Expected	User authenticated. Dashboard displayed.	
Actual Result	User logged in successfully and dashboard loaded.	
Test Script	`login("staff","staff"); openDashboard();`	

TC_002	Invalid Login Validation	PASS
Objective	Validate incorrect credential handling.	
Input	Username: test | Password: wrong123	
Expected	Login denied. Error message displayed.	
Actual Result	Invalid credentials rejected successfully.	
Test Script	`login("test","wrong123");`	

TC_003	Project Creation	PASS
Objective	Validate project creation functionality.	
Input	Project Name: User Management Service	
Expected	Project created and stored successfully.	
Actual Result	Project stored and displayed in workspace.	
Test Script	`createProject("User Management Service"); saveProject();`	

TC_004	Collection Upload	PASS
Objective	Validate Postman Collection import.	
Input	File: collection.json	
Expected	Collection imported and displayed.	
Actual Result	Collection parsed and displayed successfully.	
Test Script	`uploadCollection("collection.json"); parseCollection();`	

TC_005	Framework Configuration	PASS
Objective	Validate migration settings configuration.	
Input	Library: Requests | Base URL: https://api.example.com	
Expected	Framework configuration applied successfully.	
Actual Result	Configuration saved successfully.	
Test Script	`setLibrary("requests"); setBaseUrl("https://api.example.com"); saveConfiguration();`	

TC_006	Pytest Suite Generation	PASS
Objective	Validate AI-powered pytest code generation.	
Input	Imported Collection	
Expected	Pytest code generated and displayed.	
Actual Result	Pytest code generated successfully.	
Test Script	`compilePytest(); generateCode();`	

TC_007	Generated Code Verification	PASS
Objective	Validate structure of generated code.	
Expected	Code contains imports, test functions, assertions, request logic.	
Actual Result	Generated code structure verified successfully.	
Test Script	`openGeneratedCode(); verifyAssertions();`	

TC_008	AI Analysis Module	PASS
Objective	Validate AI-powered analysis and recommendations.	
Input	Generated Pytest Code	
Expected	AI recommendations generated.	
Actual Result	AI analysis displayed successfully.	
Test Script	`openAIAnalysis(); generateReport();`	

TC_009	Execution Center Validation	PASS
Objective	Validate test execution process.	
Input	Generated Test Suite	
Expected	Execution completed. Passed: 12, Failed: 0, Success: 100%.	
Actual Result	Execution completed successfully with all tests passing.	
Test Script	`runTestSuite(); showResults();`	

TC_010	Audit Log Generation	PASS
Objective	Validate audit logging functionality.	
Input	User Login, Project Creation, Pytest Generation	
Expected	Activities recorded successfully.	
Actual Result	Audit logs generated successfully.	
Test Script	`trackActivity(); saveAuditLog();`	

TC_011	Login History Tracking	PASS
Objective	Validate login history recording.	
Expected	Login history updated.	
Actual Result	History recorded successfully.	
Test Script	`recordLogin(); saveHistory();`	

TC_012	Staff Role Access	PASS
Objective	Validate Staff user permissions.	
Input	Role: Staff	
Expected	Staff modules accessible.	
Actual Result	Access granted successfully.	
Test Script	`loginAsStaff(); accessProjects(); accessCollections();`	

TC_013	Administrator Access	PASS
Objective	Validate Administrator permissions.	
Input	Role: Admin	
Expected	Administrative modules accessible.	
Actual Result	Administrator permissions verified successfully.	
Test Script	`loginAsAdmin(); openAuditLogs(); openLoginHistory();`	

TC_014	Unauthorized Access Prevention	PASS
Objective	Validate role-based access restrictions.	
Input	Role: Staff | Attempt: Admin Access	
Expected	Access denied.	
Actual Result	Unauthorized access blocked successfully.	
Test Script	`loginAsStaff(); accessAdminPanel();`	

4. Test Execution Summary

Test Case ID	Test Scenario	Status
TC_001	User Login Validation	**PASS**
TC_002	Invalid Login Validation	**PASS**
TC_003	Project Creation	**PASS**
TC_004	Collection Upload	**PASS**
TC_005	Framework Configuration	**PASS**
TC_006	Pytest Suite Generation	**PASS**
TC_007	Generated Code Verification	**PASS**
TC_008	AI Analysis Module	**PASS**
TC_009	Execution Center Validation	**PASS**
TC_010	Audit Log Generation	**PASS**
TC_011	Login History Tracking	**PASS**
TC_012	Staff Role Access	**PASS**
TC_013	Administrator Access	**PASS**
TC_014	Unauthorized Access Prevention	**PASS**

5. Feature Coverage Matrix

Feature	Coverage Status
Authentication	✔ **Covered**
RBAC	✔ **Covered**
Project Management	✔ **Covered**
Collection Upload	✔ **Covered**
Framework Configuration	✔ **Covered**

AI Pytest Generation	✔ Covered
Generated Code Verification	✔ Covered
AI Analysis	✔ Covered
Execution Center	✔ Covered
Audit Logs	✔ Covered
Login History	✔ Covered

6. Test Result Summary

Metric	Value
Total Test Cases	14
Passed	14
Failed	0
Blocked	0
Success Rate	100%

7. Conclusion

The Pytestify Studio platform was successfully tested across all major modules including authentication, role-based access control, project management, collection import, AI-powered pytest generation, execution monitoring, audit logging, and login history tracking. All 14 test cases passed with a **100% success rate**. The application demonstrated stable functionality across normal, happy-path, and access-control scenarios, confirming that the system is functioning as intended and is suitable for deployment and demonstration purposes.